

Res-TranBiLSTM: An intelligent approach for intrusion detection in the Internet of Things

Shiyu Wang^{a,*}, Wenxiang Xu^a, Yiwen Liu^b

^a Research Institute of China Telecom, Shanghai 200120, China

^b College of Information Science and Technology, Donghua University, Shanghai 201620, China

ARTICLE INFO

Keywords:

Intrusion detection systems
Cybersecurity
IoT networks
Cyberattacks
Transformer
BiLSTM
Feature fusion

ABSTRACT

The Internet of Things (IoT), as the information carrier of the Internet and telecommunications networks, is a new network technology comprising physical entities embedded with electronic components, software and sensors, and characterized by strong complexity and openness. With the massive amount of data, the occurrence of network intrusion is also increasingly frequent, involving industrial control systems, IoT devices, mobile security, cloud services, and telecommunications services. With the diversification and intelligence of cyberattack behaviors, traditional intrusion detection systems (IDSs) face problems—such as insufficient feature extraction and inaccurate model classification—when faced with high-dimensional features and nonlinear massive data. Due to their powerful data representation learning ability, deep learning methods save substantial time in processing high-dimensional and complex intrusion data. On this basis, we propose an intrusion detection model using ResNet, Transformer and BiLSTM (Res-TranBiLSTM) that takes into account both the spatial and temporal features of network traffic. We use the Synthetic Minor Overriding Technique (SMOTE) – Edited Nearest Neighbor (ENN) method to alleviate the degree of data imbalance. In addition, we respectively establish a spatial feature extraction model based on ResNet and a temporal feature extraction model based on Transformer and BiLSTM to extract spatial features and temporal features parallelly. Finally, spatiotemporal features are included to achieve attack detection and classification. Further, simulation experiments are conducted using the public data sets NSL-KDD and CIC-IDS2017. The experiments are implemented using Python programming language and Pytorch framework. The results reveal that the performance of our proposed model is better than that of other models, with accuracy reaching 90.99%, 99.15% and 99.56%, on NSL-KDD dataset, CIC-IDS2017 dataset and MQTTset dataset, respectively. It increased the detection accuracy by about 1%-10% on NSL-KDD dataset and about 0.2%-10% on CIC-IDS2017 dataset, and about 1%-10% on MQTTset dataset. These results demonstrate that this method is effective in constructing and optimizing large-scale IDS in the IoT environment.

1. Introduction

With the development of technologies such as the Internet of Things (IoT), cloud computing, and 5 G, as well as their increasing application in various industries, network traffic is becoming increasingly complex and cyberattacks are increasing. In April 2022, the Spring4Shell zero-day vulnerability was exposed, particularly affecting the software supplier industry. According to data from Gartner, as Web applications host increasingly important services, over 75% of attacks are directed against Web applications. Therefore, application system security is also facing unprecedented challenges. In response to the increasing diversity, novelty, and complexity of cyberattack methods, the intrusion detection

system (IDS) can be used to monitor and detect the behavior of attackers in advance and timely countermeasures can be taken to ensure the security of the IoT, cloud computing environments, and application systems [1,2].

IoT has the characteristics of complex architecture, large number of terminals, and limited computing power. How to detect security issues as early as possible, judge and protect them in real-time, and handle them in a timely manner has become a challenging issue. At present, cutting-edge solutions include high-performance intrusion detection methods [3–6], suitable deployment methods [7], and security scheduling services [8–11]. Among them, the applicability and efficiency of intrusion detection methods are the foundation.

* Corresponding author.

E-mail address: wangsy37@chinatelecom.cn (S. Wang).

<https://doi.org/10.1016/j.comnet.2023.109982>

Received 4 April 2023; Received in revised form 26 July 2023; Accepted 11 August 2023

Available online 12 August 2023

1389-1286/© 2023 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

The existing intrusion detection technologies include rule-based intrusion detection, statistics-based intrusion detection, log-based intrusion detection, and machine learning-based (ML-based) intrusion detection. The rule-based intrusion detection method matches the characteristic value of network traffic with a set rule value to determine whether or not it is an attack. This method has high detection efficiency, but requires frequent updating of the rule base; moreover, it cannot detect unknown attacks [12,13]. The statistics-based intrusion detection method detects intrusion behavior by constructing a statistical distribution of intrusion patterns. This method does not need to update the rule base but does consume a large number of computing resources, which leads to certain deficiencies in processing big data [13]. The intrusion detection method based on log analysis is to determine the abnormal behavior in the log according to different types of attack behavior by analyzing the system log. This method can identify certain attacks outside the rule base, but the results are usually obtained by analyzing logs after attack events occur, which implies a certain lag. Further, the intrusion detection technology based on ML can automatically identify attack behavior by building an ML model. This method has high detection accuracy and can defend against unknown attacks; thus, the ML algorithm is widely used in the intrusion detection process [13,14].

With the development of artificial intelligence technology, deep learning has made new breakthroughs in the field of computer vision. Deep learning technology has also been applied to other fields, such as natural language processing (NLP), speech recognition, machine translation, etc. [15,16] In terms of intrusion detection, traditional detection methods are no longer sufficient to cope with increasingly complex attacks. Although ML technology has a certain ability to detect unknown attacks, feature extraction depends on human experience, which is relatively cumbersome [17]. Further, researchers have begun to apply deep learning techniques to network intrusion detection, which can automatically extract network traffic features and identify intrusion behaviors. Moreover, deep learning techniques have better detection performance than the ML model; thus, a combination of deep learning and intrusion detection is favored by researchers.

Therefore, we adopt deep learning techniques, combining the advantages of several networks—such as the convolutional neural network (CNN), long short-term memory (LSTM) [18], and transformer—to improve the performance of intrusion detection tasks.

The main novelty of this work is: (1) We propose a new algorithm for temporal feature extraction of IoT traffic data, which introduces multi-head attention mechanism before BiLSTM to focus on the relationship among different features as well as between local features and global features; (2) We use ResNet to extract spatial features of IoT traffic data, replacing traditional manually designed features; (3) We also use dataset which is created with Message Queue Telemetry Transport (MQTT) protocol to verify the generalization of our proposed model. For IoT traffic data, we comprehensively consider temporal and spatial features to achieve the classification task, and achieve more accurate IoT intrusion detection through deployment at the fog layer.

This paper makes the following main contributions to the literature:

- (1) We analyze the importance of spatial and temporal features of network traffic information in intrusion detection tasks and summarize related studies.
- (2) We propose a Res-TranBiLSTM model that considers both the spatial and temporal features of network traffic. We use the Synthetic Minor Overriding Technique (SMOTE) – Edited Nearest Neighbor (ENN) method to alleviate the degree of data imbalance and extract spatial features and temporal features in parallel to achieve intrusion detection.
- (3) The Res-TranBiLSTM model consists of two key components: the spatial feature extraction module with ResNet and temporal feature extraction module with TranBiLSTM. In particular, TranBiLSTM introduces the multi-head self-attention in

Transformer, thereby increasing the attention among different features as well as between local features and global features. Additionally, the bi-directional long short-term memory (BiLSTM) network is used to preserve temporal features of the network traffic information, thereby making up for the defect that temporal features cannot be extracted from CNN.

- (4) We present a comprehensive evaluation of the proposed method on the NSL-KDD, CIC-IDS2017 and MQTTset data sets. We implement simulation experiments using Python programming language and Pytorch framework. Furthermore, we implement several comparison experiments. According to the experimental data, our proposed model outperforms all other approaches. It increased the detection accuracy by about 1%–10% on NSL-KDD dataset, about 0.2%–10% on CIC-IDS2017 dataset, and about 1%–10% on MQTTset dataset. The results also verify the effectiveness of constructing and optimizing large-scale IDS in the IoT environment.

The remainder of this paper is organized in the following manner. Section 2 provides the background of IDS techniques and analyzes several existing studies on a similar topic. Section 3 describes the methodology for the proposed Res-TranBiLSTM model. Section 4 provides the evaluation methodology and analyzes the experimental results. Section 5 presents the conclusions and discusses future research directions.

2. Background and related works

2.1. Convolutional neural networks and transformer techniques

Convolutional neural networks (CNNs) play an important role in image classification tasks. In 1998, LeCun proposed the LeNet-5 [19], which is trained using the BP algorithm, which is the earliest CNN model. However, in the following years of research, the effectiveness of CNNs in practical tasks was far inferior to that of traditional ML algorithms, and training was also very difficult. Until 2012, Hinton proposed AlexNet [20], and for the first time subverted the field of image recognition using CNN. Subsequently, the development of CNN also ushered in a climax period, with endless network structures such as VGGNet [21], GoogLeNet [22], and ResNet [23] gradually playing an important role in various image recognition tasks. Compared with the manual feature extraction of the ML model, the CNN model has outstanding advantages since it can extract image features automatically. Therefore, the CNN model is also widely used in NLP, and speech recognition. In addition, inspired by the ability of the CNN model to automatically extract features for classification, researchers also apply CNN models to process one-dimensional data. For example, Xiao et al. [24] utilized CNNs to detect network intrusion behaviors. Moreover, experiments revealed that CNN-based models had better detection performance than other ML models.

Further, the Transformer [25] model was proposed by the Google team in 2017. Due to its ability to capture the long-distance dependency in an input sequence, in recent years, the transformer and its variants have been widely used in NLP tasks—such as machine translation, speech recognition, text summarization, and text completion—and have achieved great success. Therefore, an increasing number of researchers attempt to introduce the Transformer model into the field of computer vision to solve problems of image classification, image recognition, image generation, etc., which promotes the development of the image field. Moreover, inspired by Transformer's ability to process ordered data sequences, Huang et al. [26] built a log-based anomaly detection model by using a variant of Transformer and proved its robustness to unstable log data. Wu et al. [27] detected network abnormal traffic based on Transformer. Experiments have revealed that this method has better detection performance than recurrent neural networks (RNNs) [28], LSTM, etc.

2.2. Intrusion detection using deep learning techniques

The related works for intrusion detection using deep learning techniques are listed in Table 1, focusing on data set, method, feature type, description of advantages and disadvantages, and classification types.

Zhang et al. [32] proposed a novel intrusion detection network based on deep learning—parallel cross convolutional neural network (PCCN). By fusing the flow features learned from two branch CNNs, fewer samples can be used to better learn the flow features in order to improve the detection effect of unbalanced abnormal traffic. An improved original flow feature extraction method was proposed to extract multi-class flow features simultaneously to speed up the convergence rate of the network. Xu et al. [34] proposed a DNN architecture FC—Net based on meta-learning to solve the problem that traditional intrusion detection systems based on supervised learning require a large number of samples for training while malicious samples in specific security scenarios are always rather limited. By first learning a pair of feature maps for classification from a pair of network traffic samples, the feature maps were then compared to determine whether they belong to the same traffic type. The proposed method is not limited to a specific data set and is universal. Hu et al. [35] processed unbalanced data sets by the adaptive synthetic sampling (ADASYN) data augmentation algorithm, and designed a CNN model based on chan-nel splitting to detect intrusion. Andresini et al. [38] proposed a multi-stage IDS by combining nearest neighbor search, clustering and CNNs. Wang et al. [39] converted Industrial Control System (ICS) stream data into two-dimensional gray images; on the basis of modifying the existing remaining CNN structure, they fully utilized the features of CNN to extract two-dimensional data features and used transfer learning technology to solve the problem of

long training time of the deep learning model, thereby meeting the real-time detection requirements of IDS. Fatani et al. [40] proposed a feature extraction method based on CNN to automatically extract relevant features from the input data set, and proposed a transient search optimization algorithm using the operators of differential evolution (TSODE) to further improve the performance of the TSO algorithm in feature selection by using the DE difference operator to enhance the search stage of the traditional TSO algorithm. Sun et al. [41] proposed an IDS model of deep CNN based on residual units using SMOTE to process unbalanced data sets. Yang et al. [43] proposed a novel framework of IDS on the basis of few-shot learning using CNN. These papers only used the CNN network structure to extract the spatial features of network traffic, without considering that an attack launched by attackers may be closely related to time.

Xu et al. [29] proposed a new intrusion detection method based on bidirectional Gate Recurrent Unit (GRU). By using bidirectional GRU to extract features and MLP to make classification decisions, this method was found to have a higher detection rate and a lower false positive rate, particularly for DOS attacks. Moreover, it also proved that GRU is an effective simplification and improvement of LSTM. Wang et al. [31] proposed a LSTM tree (LSTMTree) model with secondary detection ability based on the LSTM. By first using the LSTM model to distinguish normal and attack samples, and then training the decision tree model based on the normal samples identified by LSTM, it can be further distinguished whether the normal samples contain attack samples to solve the problem of a high false negative rate of RNN intrusion detection methods. Further, Sivamohan et al. [36] proposed a cyclic neural network based on BiLSTM to realize efficient intrusion detection. Two groups of experiments were conducted to determine the optimal

Table 1
Summary of papers with similar topics.

Refs.	Year	Dataset	Method	Feature	Description	Classification
[29]	2018	KDD99, NSL-KDD	GRU	SF	Bi-GRU and MLP are used to improve detection performance. Data imbalance is not considered.	MC
[30]	2019	CIC-IDS2017, CTU dataset	CNN+LSTM	RT	LeNet-5 and LSTM are combined to improve the accuracy of classification. Network structure is simple.	MC
[31]	2019	NSL-KDD	LSTM	SF	LSTMTree model is proposed to further detect malicious attacks. Data imbalance is not considered.	BC / MC
[32]	2019	CIC-IDS2017	CNN	RT	PCNN network architecture is proposed to use fewer samples to flow features. Only spatial features can be extracted.	MC
[33]	2020	CSIC-2020, CIC-IDS2017, KF-ISAC	CNN+LSTM	SF	CNN and LSTM fusion model based on SFL is applied to real scenes. False alarm rate is high.	BC
[34]	2020	CICIDS2017, ISCX2012	CNN	RT	FC-Net is proposed to solve the problem of limited samples. It is unable to detect specific attack types.	BC
[35]	2020	NSL-KDD	CNN	SF	A novel CNN model based on channel splitting is proposed. The recognition accuracy of small samples is not satisfactory.	MC
[36]	2021	CIC-IDS2017	LSTM	SF	BiLSTM is used to improve the accuracy of network detection. It cannot detect specific attack types.	BC
[37]	2021	CICIDS2017	LSTM	SF	LSTM is used to detect intrusion behavior. Training time is too long.	BC
[38]	2021	CIC-IDS2017	CNN	SF	A 2D CNN architecture with nearest cluster is proposed. The CNN structure is rather simple.	BC
[39]	2021	KDDCUP99, Gas Pipeline dataset	CNN	SF	Modified CNN structure and transfer learning is used to shorten training time. It is unable to detect specific attack types.	BC
[40]	2021	KDDCup-99, NSL-KDD, BoT-IoT, CICIDS2017	CNN	SF	TSODE algorithm is used to select the best features. Only the spatial feature is considered.	BC / MC
[41]	2022	NSL-KDD	CNN	SF	A deep CNN based on residual units is proposed. The data set is old and simple.	MC
[42]	2022	NSL-KDD, BoT-IoT	CNN+LSTM	SF	CNN and LSTM are used to improve detection performance. Multi-classification is not considered.	BC
[43]	2022	CIC-IDS2017, UNSW-NB15, KDDCUP99	CNN	SF+RT	An integrated intrusion detection framework based on few-shot learning is proposed. Only the spatial feature is considered.	MC
[44]	2023	CIC-IDS2017	LSTM	SF	A stacked LSTM model with a robust pre-processing technique is proposed. Only the temporal feature is considered.	BC
[45]	2023	NSL-KDD, UNSW-NB15	LSTM	SF	A diffusion model is proposed to equilibrate the rare class data set. The training speed is slow.	MC
[46]	2023	CIC-IDS2017, UNSW-NB15	CNN+LSTM	SF	IGAN, CNN, and LSTM is used to detect the attacks. The CNN structure is rather simple.	BC / MC

Note: SF: Statistic Features; RT: Raw Traffic; MC: Multi-Classification; BC: Binary Classification.

network hyperparameters and use the optimal hyperparameters for intrusion detection, thereby determining the processing time of the algorithm and improving the accuracy of attack classification. Alsaibani et al. [37] proposed a BiLSTM to identify malicious traffic in a network; they utilized the optimizer, activation function, and learning rate as the training research variables to conduct several sets of scene experiments with different training parameters in order to obtain the highest accuracy of intrusion detection. João et al. [44] proposed a stacked LSTM model with a robust pre-processing technique for intrusion detection, and achieve an accuracy of 99.01% in binary classification task. Tang et al. [45] proposed an Industrial Cyber-Physical Systems (ICPS) IDS based on the diffusion model using BiLSTM. Although these papers considered that attacks may be temporal, the extracted features may not be comprehensive.

By combining the improved LeNet-5 and LSTM structures, Zhang et al. [30] simultaneously learned the temporal and spatial features of original traffic data and designed a reasonable network cascade method to simultaneously train the proposed hierarchical network. This method can achieve high accuracy. Kim et al. [33] proposed a fusion model of an optimal CNN-LSTM network based on spatial feature learning (SFL) to fully extract the features of real-time HTTP traffic and successfully applied payload level deep learning technology to high-performance computing environments; their study indicated that scalability and precision were also important indicators for applying deep learning to realistic scenarios. Further, Deore et al. [42] proposed a deep LSTM network intrusion detection method based on the Chimp Chicken Swarm Optimization (ChCSO) method. By using CNN to extract features and deep LSTM to determine intrusion behavior, as well as using the designed ChCSO method for training, this method has better detection accuracy and is easily extendable. Rao et al. [46] used the Imbalanced Generative Adversarial Network (IGAN) technique to solve the class imbalance problem, and adopted the CNN and the hybrid LSTM ensemble model approach to achieve the intrusion detection. These papers all used CNN and LSTM fusion models to extract the spatial and temporal features of network traffic, and the detection effect is good. However, they either did not carry out multi-classification verification or did not solve the problem of data imbalance.

From the extant research papers we examined, we noticed that on the

one hand, certain researchers preprocess network traffic packets into image form and use CNN to learn their spatial features for classification, thereby detecting abnormal traffic; on the other hand, certain researchers treat network traffic packets as sequences, using RNN and LSTM to process them. Further, although certain researchers have combined spatial and temporal features for intrusion detection, the network structures used in existing studies are relatively simple.

3. Methodology

The reference architecture of the proposed intrusion detection model in IoT is depicted in Fig. 1.

Fog computing is a computing paradigm that can provide computing and storage services between terminal devices and cloud services to compensate for the shortcomings of cloud computing. It can solve problems such as difficulty in fast response and poor device flexibility in cloud computing [47].

The architecture of fog computing consists of three layers, namely edge layer, fog layer, and cloud layer. The edge layer consists of many edge nodes, including many IoT devices. The fog layer consists of many fog nodes. Fog nodes represent heterogeneous components deployed in edge networks of fog environments, including intelligent gateways, routers, switches, and specific fog servers. The cloud layer includes large cloud servers and well configured data centers. It has powerful computing resources and can easily aggregate, train and store big data.

The mode combining fog computing and deep learning has been proven effective in supporting low latency intrusion detection tasks in IoT [48]. Therefore, this study fully utilizes the characteristics of fog computing and implement the proposed Res-TranBiLSTM model by using the fog nodes and the cloud servers in a combining way.

As depicted in Fig. 1, the training process of the Res-TranBiLSTM model is completed by the cloud server. Then, the trained model is deployed on fog nodes. Specifically, the implementation process is as follows. Firstly, fog nodes collect IoT traffic data from edge nodes. Secondly, when data is transmitted to the fog node, it performs traffic data preprocessing operations and calls the trained detection model to classify the data without any communication with the cloud backend, thereby reducing detection response time. Once an activity is identified

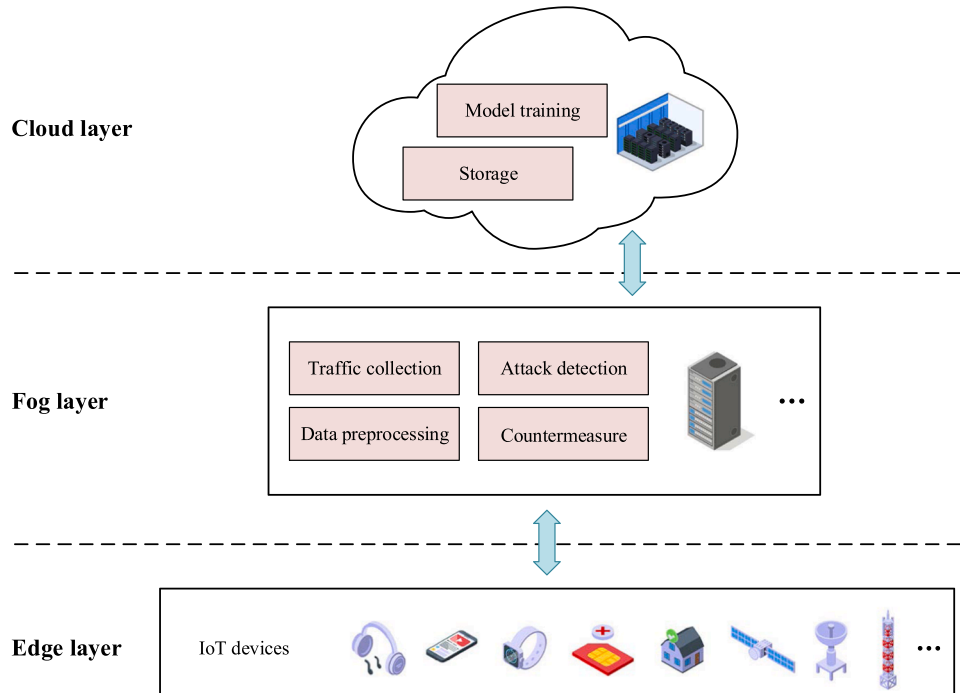


Fig. 1. Architecture of the proposed intrusion detection approach in IoT.

as an attack, the information provided in the activity will be forwarded to the cloud backend. Each fog node is responsible for diagnosing the relevant areas of the IoT network.

3.1. Overview of the Res-TranBiLSTM model

The Res-TranBiLSTM model proposed in this paper consists of three stages: data preprocessing, spatiotemporal feature extraction, and intrusion attack classification. The spatiotemporal feature extraction stage is implemented by a parallel combination of the spatial feature extraction module with ResNet and temporal feature extraction module with TranBiLSTM. The entire workflow is depicted in Fig. 2.

As depicted in Fig. 2, the original features need to be reshaped and encoded using multilayer perceptron (MLP) to meet the input requirements for the model; these are then input into ResNet and TranBiLSTM for parallel spatial and temporal feature extraction, respectively. ResNet is used to learn spatial features by adding residual connections to the network, thereby helping to maintain the information flow throughout the network and preventing the gradient from disappearing. In the Tran-BiLSTM model, several Transformer [25] encoders are used to extract richer feature information through multi-head self-attention [25]; thereafter, several BiLSTM blocks are used to mine the temporal relationship between pre- and post-features to obtain temporal feature representations among features. Finally, the obtained spatial and temporal features are fused through the concatenation (concat) operation and then sent to the classification module. The final classification is performed through several fully connected layers using the softmax function.

3.2. Data preprocessing

Data preprocessing includes three steps: (1) data digitization and encoding, (2) class imbalance processing, and (3) data normalization processing.

3.2.1. Data digitization and encoding

The binarization technique called the one-hot encoding method is used to convert the original features into numerical values, enabling the subsequent networks to process the features.

In addition, for the spatial feature extraction module, it is necessary to reshape the 1D features into 2D features, so that local features can be extracted in the form of grayscale images. The specific processing methods vary depending on different data sets. For the temporal feature extracted module, similar to the word embedding layer in NLP, an MLP

layer is used to encode data for each feature. Thereafter, the features are amplified to map to different subspaces, thereby extracting richer features and meeting the input dimensions required by the model. Simultaneously, the parameters of the MLP layer can be dynamically adjusted during training.

3.2.2. Class imbalance processing

In real life, the number of samples recorded for normal traffic exceeds that recorded for abnormal traffic. In this case, IDS pays more attention to normal traffic records to maximize the overall classification accuracy. However, the original intention of developing IDS is to focus more on abnormal traffic. Therefore, data imbalance can seriously affect the performance of IDS. It is found that there is a serious category imbalance in the public data set of existing intrusion detection tasks. To solve this problem, we use the SMOTE [49] algorithm.

Specifically, for a minority class sample x_i , K-nearest neighbor (KNN) method is used to find the k minority samples closest to x_i . Then, one of the KNN points is randomly selected, and a new sample is generated using Eq. (1):

$$x_{new} = x_i + (\hat{x} + x_i)\delta, \quad (1)$$

where $\hat{x}$ is the selected neighbor point and $\delta \in [0, 1]$ is a random number.

However, the disadvantage of the SMOTE algorithm is that the generated minority class samples are apt to overlap with the surrounding majority class samples, which can affect the efficiency of the classifier. The data cleaning technology can handle overlapping samples. In the proposed model, the ENN [50] algorithm is used to delete data that overlaps with a majority of the class samples.

The ENN algorithm completes the downsampling operation by eliminating noise samples. It defines noise samples as samples that belong to different classes among the nearest neighbor samples found by the samples based on Euclidean distances.

3.2.3. Data normalization processing

In order to eliminate data differences caused by different dimensions, normalization processing is performed after data digitization and encoding. The min-max normalization method is adopted, which can alleviate the differences caused by different dimensions and unify the values. The specific formula is depicted in the form of Eq. (2):

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (2)$$

where x is the original feature, x_{min} and x_{max} denote the maximum and

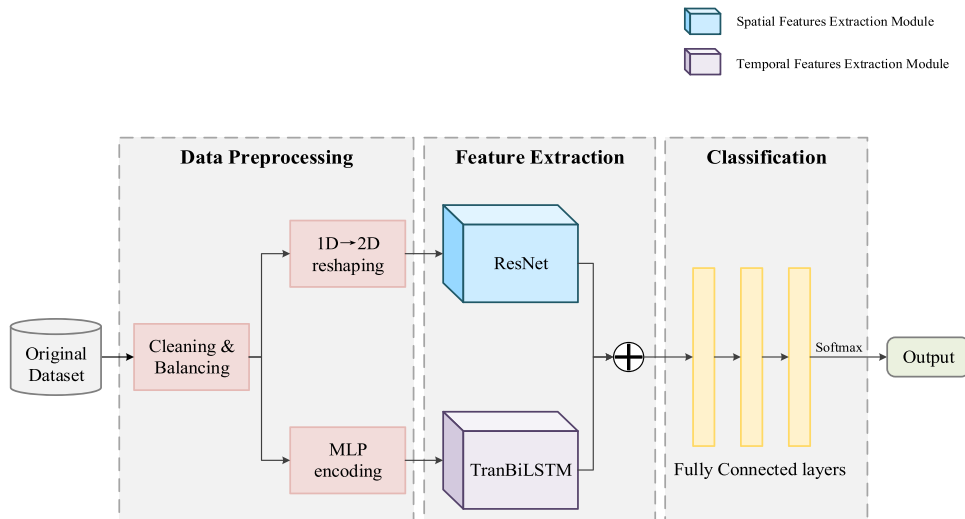


Fig. 2. Overall structure of the proposed Res-TranBiLSTM model.

the minimum values for feature x , respectively, and x_{norm} is the result of normalization processing.

3.3. Spatial feature extraction module with ResNet

The network structures used in existing research on extracting spatial features of traffic data are relatively simple. Therefore, they cannot highlight the advantages of CNN in local feature extraction. Therefore, we adopt ResNet with deeper layers and a more complex structure. Its residual structure can greatly improve the learning ability of the network, thereby making the classification effect optimal.

The RNN ResNet is one of the network models developed by He et al. [23] in 2016. The structure diagram is depicted in Fig. 3. This network allows a new layer not to merely replicate the existing shallow layer but to fit some residual mapping through a model. Compared to classical CNNs, ResNet has the advantage of changing the learning objectives and replacing direct output values with residual values that are easier to optimize through learning. This approach not only protects the integrity of data information when it is transferred between convolutional layers but also reduces the complexity of learning to a certain extent.

As presented in Fig. 3, in a residual block, a batch normalization (BN) layer is added after each convolution layer to reduce the impact of the scale of the weight value. The noise caused by normalization has the effect of regularization. The calculation formula for learning identity mapping for each residual block is presented in Eq. (3):

$$\begin{cases} X^{[l+c]} = f(h(X^{[l]}) + F(x)) \\ F(x) = \sum_{i=1}^{l+c-1} F(x_i) \end{cases}, \quad (3)$$

where $X^{[l]}$ and $X^{[l+c]}$ represent the input and output vectors of the residual block, respectively; $F(x)$ represents the residual mapping of the residual block; and c represents the number of convolution layers included in the residual block.

The design goal of this network structure is to solve the problem of deep learning training, which usually requires a considerable amount of time and is limited to a certain number of levels. The core idea is to calculate the residual between layers and add the original input and final output through skip connections. Compared to models with other architectures, ResNet has the advantage that even if the number of network layers becomes deeper, the performance of the model will not worsen. In addition, it can achieve lighter computing and better network training capabilities.

3.4. Temporal feature extraction module with TranBiLSTM

In order to enable the model to focus on important features without losing the relevance between local features, we propose a temporal feature extraction module that combines Transformer and LSTM and is called TranBiLSTM. The structure of the TranBiLSTM module is depicted in Fig. 4.

TranBiLSTM consists of a Transformer encoder and a BiLSTM block. The introduction of multi-head self-attention increases the attention among different features, as well as between local and global features, and explores the internal relevance among features. Additionally, it uses BiLSTM to preserve temporal features.

3.4.1. Transformer encoder block

The original structure of Transformer [25] consists of two parts:

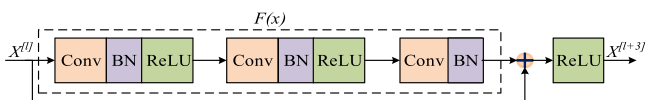


Fig. 3. Partial network structure diagram of ResNet.

encoder and decoder. Only the encoder is used in the proposed TranBiLSTM. As depicted in Fig. 4, a Transformer encoder block consists of a multi-head self-attention and a feed-forward network and utilizes residual connections to prevent gradient disappearance.

(1) Multi-head self-attention and scaled dot-product attention

The multi-head self-attention mechanism comprises multiple scaled dot-product attention. It computes three inputs **Query**, **Key**, and **Value** to obtain an output vector with attention. **Query**, **Key**, and **Value** are converted into input that conforms to the scaled dot-product attention through linear operations, and then the output of multiple heads is integrated using the concat operation through scaled dot-product attention calculations. The output is then completed through the linear layer to obtain an output with attention. The formula for calculating the multi-head self-attention and scaled dot-product attention is expressed in Eqs. (4) and (5), respectively.

$$\text{MultiHeadAtt}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \Theta(h_1, h_2, \dots, h_i)W^o \quad (4)$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \cdot \mathbf{V}, \quad (5)$$

where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ represent the three matrices **Query**, **Key**, and **Value**, respectively. d_k is the dimension of **Key**, h_i represents the output of the i -th header, Θ represents the concat operation, and W^o is the weight matrix.

(1) Feed-forward network

This network comprises a two-layered fully connected network. The first layer uses the ReLU activation function, and the second layer does not use the activation function. The formula is presented in Eq. (6):

$$\text{FNN}(x) = \max(0, xW_1 + b_1)W_2 + b_2, \quad (6)$$

where W_1 and W_2 are weight matrices, and b_1 and b_2 are bias matrices.

3.4.2. The BiLSTM block

The LSTM [18] network is an improved RNN [28] network that solves the problem of long-distance information loss in long sequences. It is used to process temporal information and solve the problems of gradient explosion and gradient disappearance in an RNN structure. It can remember valuable information and give up redundant memory. In the forward propagation process, LSTM controls the storage and interaction of information through the three gate structures of the hidden layer memory cell, including forget gate, input gate, and output gate. The internal structure of an LSTM cell is presented in Fig. 5.

The calculation formula for each parameter in Fig. 5 is presented in Eq. (7):

$$\begin{cases} f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \\ i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \\ \tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \\ C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \\ o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \\ h_t = o_t * \tanh(C_t) \end{cases}, \quad (7)$$

where σ represents the sigmoid function; W_f , W_i , W_c , and W_o are the weight matrices of forget gate, input gate, and memory cell and output gate, respectively; h_{t-1} is the output of the hidden layer at the previous time; C_t is the cell state at the current time t ; o_t is the output of the output gate at the current time t ; h_t is the output of the hidden layer at the current time t ; x_t is the input vector; b_f , b_i , b_c , and b_o are the bias terms of forget gate, input gate, and memory cell and output gate, respectively.

In a BiLSTM network, each training sequence consists of a forward-propagating LSTM network and a backward-propagating LSTM

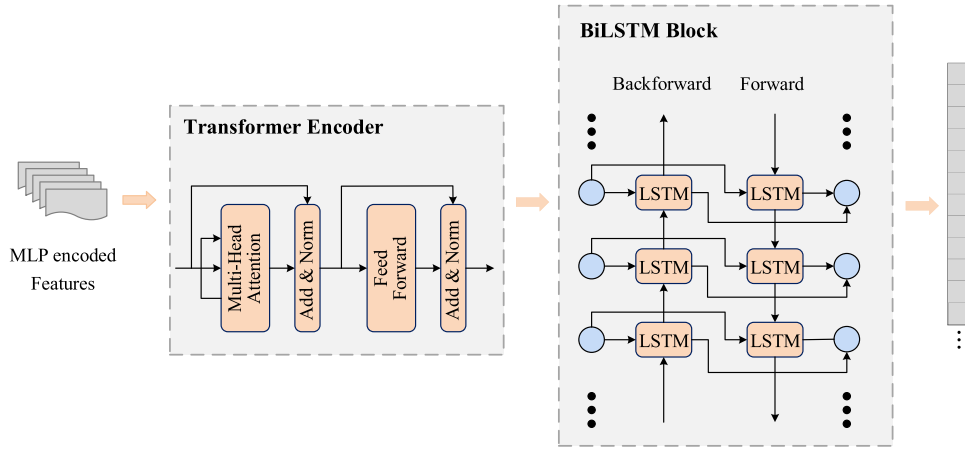


Fig. 4. The structure of the TranBiLSTM module.

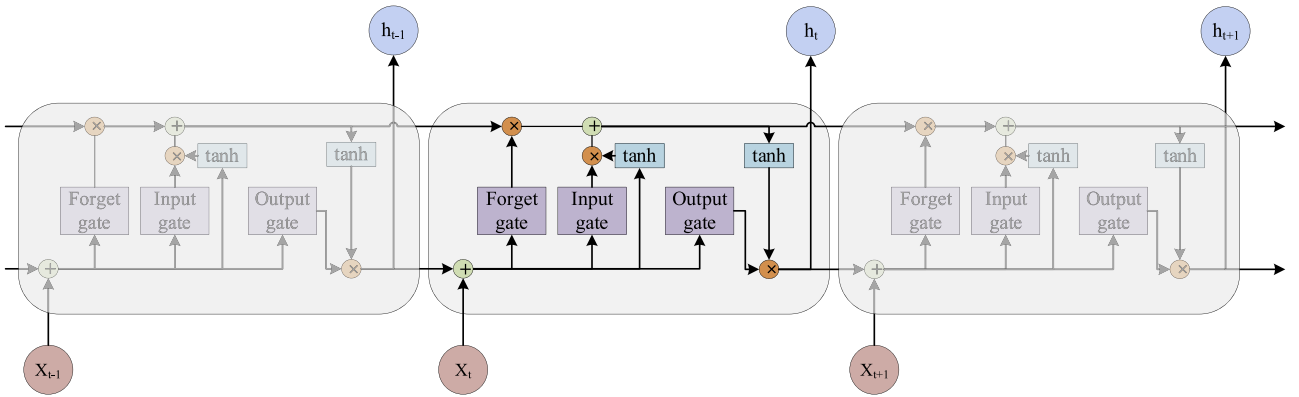


Fig. 5. The internal structure of an LSTM cell.

network. These two LSTM networks are simultaneously connected to an output layer, providing complete contextual information for each neuron in the output layer. The BiLSTM block used in this paper is presented in Fig. 4. The input of each output neuron consists of a forward LSTM layer output and a reverse LSTM layer output. The calculation formula of output is presented in Eq. (8):

$$h_t = \begin{bmatrix} \vec{h}_t, \overleftarrow{h}_t \end{bmatrix}, \quad (8)$$

where $\vec{h}_t$ and $\overleftarrow{h}_t$ represent the outputs of the forward and backward networks at time t , respectively.

4. Experiments and results

4.1. Description of data sets and data preprocessing

In this section, we describe the data sets used for experimental purposes. In this study, we utilized the NSL-KDD data set [51], the CIC-IDS2017 data set and the MQTTset data set to evaluate the performance of the proposed model.

The NSL-KDD data set [51] was used in 8% of the reviewed papers. It is a revised version of the well-known KDD'99 data set [52], which consists of four sub-data sets: KDDTest+, KDDTest-21, KDDTrain+ and KDDTrain+_20Percent. Among these, KDDTest-21 and KDDTrain+_20Percent are subsets of KDDTrain+ and KDDTest+. Each record in the data set contains 43 features, 41 of which refer to the input itself, while the last two are the label (normal or attack) and the score (severity of the input). The data set contains four different types of

attacks: denial of service (DoS), probe, user to root (U2R), and remote to local (R2L).

For the NSL-KDD data set, the SMOTE-ENN algorithm is directly used to process the original data and generate new training set and testing set. The class distribution of the original data set and the pre-processed one is presented in Table 2, and the graphical presentation is depicted in Fig. 6.

However, there are still some problems with the NSL-KDD data set. Its testing set contains classes and feature distributions that have never appeared in the training set. In other words, there are certain differences in the distribution of the training set and the testing set, thereby resulting in a decline in the accuracy of the model. Therefore, the NSL-KDD data set is not entirely suitable for verifying the performance of IDS models. In order to further verify the effectiveness of the proposed model, we also consider the CIC-IDS2017 data set.

The CIC-IDS2017 data set was developed by the Canadian Institute of Network Security at the end of 2017 by generating and capturing network traffic over a period of five days. The CIC-IDS2017 data set

Table 2
Class distribution of the NSL-KDD data set.

No.	Labels	Original Dataset			Processed Dataset		
		Train	Test	Total Portion	Train	Test	Total Portion
1	Normal	67,343	9711	51.88%	66,618	9046	19.78%
2	DoS	45,927	7458	35.95%	67,230	9638	20.10%
3	Probe	11,656	2421	9.48%	67,242	9650	20.10%
4	R2L	995	2754	2.52%	67,229	9279	20.00%
5	U2R	52	200	0.17%	67,253	9288	20.01%

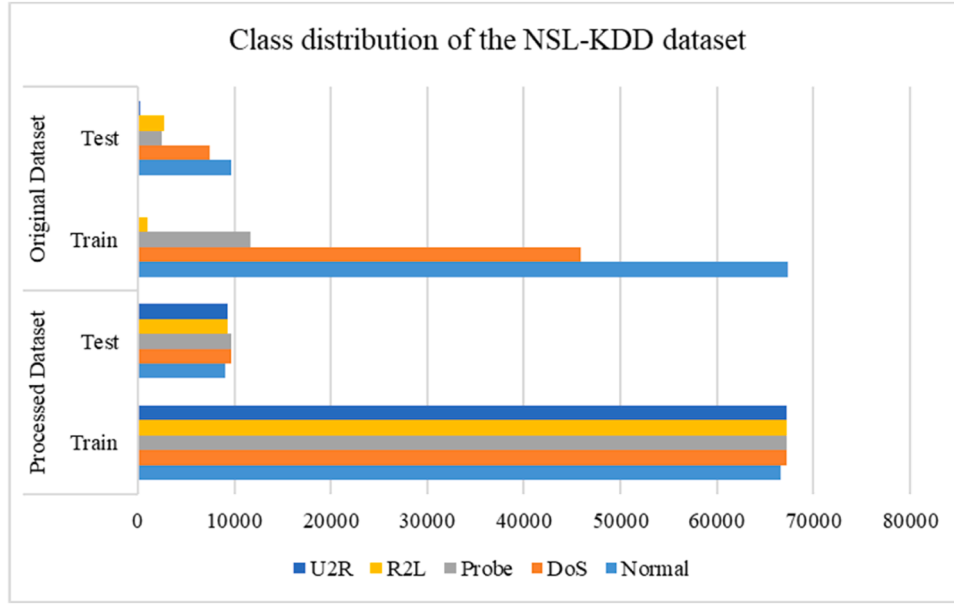


Fig. 6. Class distribution of the NSL-KDD data set.

contains the latest common attacks, which are closer to real data. The complete CIC-IDS2017 data set contains 8 files, a total of 3,119,345 instances and 83 features of 15 class labels (1 normal label and 14 attack labels). The distribution of traffic classes in the data set after data cleaning is presented in Table 3.

For the CIC-IDS2017 data set, we first merge similar labels and then uniformly filter the data under each label to avoid causing excessive data size after processing. Finally, we use the SMOTE-ENN algorithm for processing. The processed data distribution is presented in Table 4.

The graphical presentation of Tables 3 and 4 is depicted in Fig. 7.

The MQTTset [53] data set was provided by Kaggle enterprise. The traffic data was collected from 8 types of IoT devices: thermometers, light sensors, humidity sensors, smoke sensors, fan sensors, motion sensors, smart door locks, and gas detection, totaling 10 devices. The data set was built using IoT-Flock [54] tool to generate traffic data, including legitimate (normal) data and five types of attacks: Flooding Denial of Service (DoS), Brute Force Authentication (Brute Force), Malformed Data (Malformed), SlowITe and MQTT Publish Flood (Flood).

For the MQTTset data set, the SMOTE-ENN algorithm is directly used to process the original data and generate new training set and testing set. The class distribution of the original data set and the pre-processed one is presented in Table 5, and the graphical presentation is

Table 4

Class distribution of the processed CIC-IDS2017 data set.

No.	New Labels	Original Labels	Number	Portion
1	Normal	BENIGN	26,188	14.56%
2	Bot	Bot	26,137	14.53%
3	Brute Force	FTP-Parator SSH-Parator	25,916	14.41%
4	DDoS	DDoS	24,330	13.53%
5	DoS	DoS GoldenEye DoS Hulk DoS Slowhttptest DoS slowloris Heartbleed	25,890	14.40%
6	PortScan	Infiltration PortScan	25,529	14.20%
7	Web Attack	Web Attack-Brute Force Web Attack-Sql Injection Web Attack-XSS	25,838	14.37%

depicted in Fig. 8.

The features in these data sets are also preprocessed and transformed for building neural networks. The following are the results of the pre-processed features in each data set:

The NSL-KDD data set consists of 41 features. Among these, the feature protocol_type has three values: TCP, UDP, and ICMP, with binary vectors of (1,0,0), (0,1,0), and (0,0,1), respectively. The 70 attributes of the feature service and 11 attributes of the feature flag were processed in the same manner. After digitization and normalization operations, each sample changed from the original 41-dimensional feature to the 122-dimensional feature; one feature with all zeros was considered to have no impact on the classification results and was, thus, deleted. Therefore, a 121-dimensional feature vector was obtained and mapped to a matrix form of 11×11 , thereby enabling the model to better learn data features.

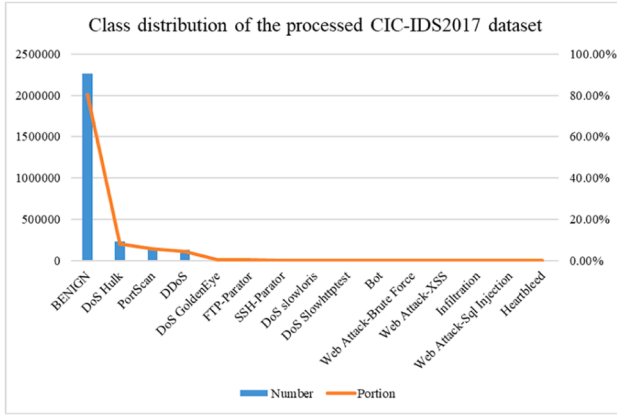
The CIC-IDS2017 data set was extracted with 84 features from the original traffic packet, using the flow feature extraction tool called CICFlowMeter [55]. After data cleaning and feature selection, only 64 features remained. These features were mapped into a matrix form of 8×8 , thereby enabling the model to better learn data features.

The original MQTTset data set consists of 60 features. Firstly, missing fields were filled in and address related features ('eth. src', 'eth. dst', 'ip. src', 'ip. dst') were removed. Secondly, the text feature 'mqtt.

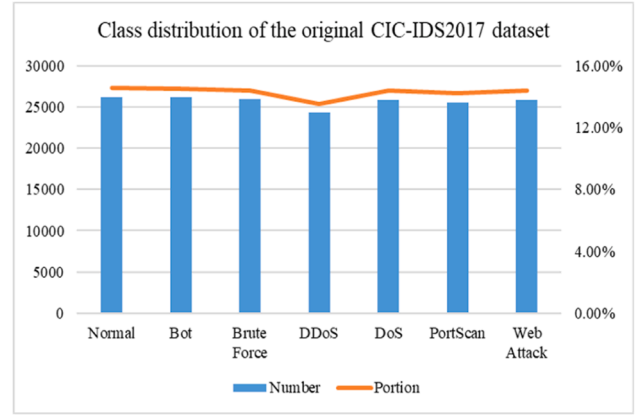
Table 3

Class distribution of the original CIC-IDS2017 data set.

No.	Labels	Number	Portion
1	BENIGN	2,260,360	80.33%
2	DoS Hulk	229,198	8.15%
3	PortScan	157,703	5.60%
4	DDoS	127,082	4.52%
5	DoS GoldenEye	10,289	0.37%
6	FTP-Parator	7894	0.28%
7	SSH-Parator	5861	0.21%
8	DoS slowloris	5771	0.21%
9	DoS Slowhttptest	5485	0.19%
10	Bot	1943	0.07%
11	Web Attack-Brute Force	1497	0.05%
12	Web Attack-XSS	648	0.02%
13	Infiltration	34	0.00%
14	Web Attack-Sql Injection	21	0.00%
15	Heartbleed	11	0.00%



(a)



(b)

Fig. 7. Class distribution of the CIC-IDS2017 data set. (a) the original data set; (b) the processed dataset.

Table 5
Class distribution of the MQTTset data set.

No.	Labels	Original Dataset		Processed Dataset	
		Number	Portion	Number	Portion
1	Normal	165,463	50.00%	15,839	20.16%
2	DoS	130,233	39.35%	15,068	19.04%
3	Brute Force	14,501	4.38%	14,501	18.46%
4	Malformed	10,924	3.30%	10,924	13.80%
5	SlowTe	9202	2.78%	11,387	14.50%
6	Flood	613	0.19%	10,838	13.69%

protoname' and label 'target' were encoded and transformed. The 'flag' was converted from hexadecimal form to Decimal form. Then, the MQTT protocol port information is given special attention to extract text features: (1) The 'mqtt. clientid', 'mqtt. passwd', and 'mqtt. msg' were extracted as text length, number of text letters, and number of special characters in the text; (2) The 'mqtt. passwd' was extracted as a 6-dimensional hash vector; (3) The 'mqtt. topic' was extracted as text length, whether it contains SYS, and whether it contains malaria. Finally, 64 features were extracted. These features were mapped into a matrix form of 8×8 , thereby enabling the model to better learn data features.

Due to the fact that the proposed model requires multiple downsampling operations in the experiment, and the size of the feature matrix is reduced by one time for each downsampling operation, the sizes of 11×11 and 8×8 cannot meet the input requirements of the model. Therefore, the feature matrices of 11×11 and 8×8 are respectively

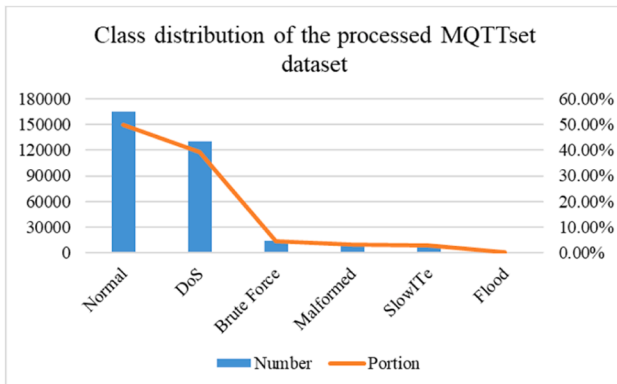
expanded to two feature matrices of 28×28 using a bicubic interpolation algorithm, which is consistent with the size of the MNIST hand-written digit data set.

4.2. Experimental setup

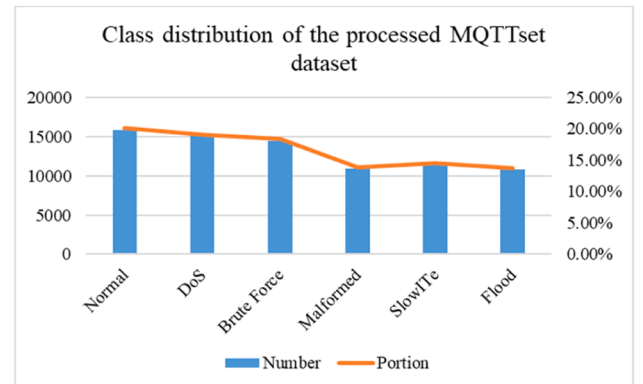
The code was executed using Python language on a computer running on Intel(R) Core(TM) i7-12700H CPU @ 2.70 GHz 4.70 GHz and 16.0 GB RAM, Windows 11 Home Basic 64bit. An NVIDIA GeForce RTX 3060 GPU was used as the accelerator. In addition, Pytorch was used as the software framework.

Optimal Res-TranBiLSTM architecture was developed by adjusting various parameters, including the size of the layers and filters of ResNet, the number of the Transformer encoder block and of the BiLSTM cell, and the kernel of the fully connected layers, etc. The optimal hyper-parameters that were selected after several experiments are explained in the following manner.

Table 6 presents the configuration of the ResNet network structure parameters in the Spatial Feature Extraction Module, where ReLU is selected as the activation function for each layer. Due to the small input size of the image itself, we adopt a convolutional kernel with a receptive field of 3×3 , which not only ensures more precise learning of spatial features but also reduces the number of parameters and improves convergence speed. The model consists of a pre-convolution layer, a total of 16 convolution layers from among 4 residual blocks, and a fully connected layer. The final output dimension of the full connection layer is 128, thereby facilitating the integration with the temporal features extracted by the temporal feature extraction module.



(a)



(b)

Fig. 8. Class distribution of the MQTTset data set. (a) the original data set; (b) the processed dataset.

Table 6
Structure parameters of the spatial feature extraction module.

Layer name	Output size	Content
conv1	28×28	3×3 , 64, <i>stride</i> : 1
conv2_x	14×14	3×3 max pooling, <i>stride</i> : 1
		$\begin{bmatrix} 3 \times 3, & 64 \\ 3 \times 3, & 64 \end{bmatrix} \times 2$
conv3_x	7×7	$\begin{bmatrix} 3 \times 3, & 128 \\ 3 \times 3, & 128 \end{bmatrix} \times 2$
conv4_x	4×4	$\begin{bmatrix} 3 \times 3, & 256 \\ 3 \times 3, & 256 \end{bmatrix} \times 2$
conv5_x	2×2	$\begin{bmatrix} 3 \times 3, & 512 \\ 3 \times 3, & 512 \end{bmatrix} \times 2$
		max pooling, FC-1(128)

The network structure of the temporal feature extraction module on each layer is presented in Table 7. And Table 8 presents the hyperparameter configuration of the temporal feature extraction module. We use the last output of the forward LSTM and the last output of the backward LSTM for the concatenation and as the input to the next layer, and sets an input dimension that is twice the output dimension to minimize the complexity of the model.

The network structure of the classification module on each layer is presented in Table 9. Due to the multi-class classification problem, the activation function of the last layer is softmax. And Table 10 presents the hyperparameter configuration of the classification module.

We use the Adam optimizer to train our model. For the NSL-KDD data set, we use the preprocessed training set and test set in Table 2. For the CIC-IDS2017 data set, we divide the data in Table 4 into a training set and test set in an 8:2 ratio. For the MQTTset data set, we divide the data of the processed dataset in Table 5 into a training set and test set in an 8:2 ratio. To avoid model overfitting, we use early-stopping criteria to train Res-TranBiLSTM—that is, we stop the training process once the validation accuracy begins decreasing.

4.3. Evaluation metrics

In order to fully evaluate the classification performance of the Res-TranBiLSTM model, accuracy, precision, recall, and F1-score are used as model evaluation metrics in the experiments.

Confusion matrix is a classic visual evaluation tool used in ML. The columns of the confusion matrix represent predicted class results, and the rows represent actual class results. It lists all possible scenarios for classification problems. Taking the binary classification problem as an example, the dimension of the confusion matrix is 2×2 , as shown in Fig. 9.

For multiclassification problems, when evaluating the classification of a certain traffic class, this traffic class is considered a positive sample, while the remaining traffic classes are considered negative samples. The confusion matrix can be used to define the metrics of classification performance, such as accuracy, precision, recall, and F1-score.

- (1) Accuracy: It is the proportion of the number of correctly classified samples in all samples.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (9)$$

Table 7
Network structure of the temporal feature extraction module.

Block	Layer	Output	Processing Fuction
MLP encoding	FC-2	(len_seq, 16)	ReLU + Dropout
	FC-3	(len_seq, 32)	ReLU + Dropout
Transformer encoder	Encoder	(len_seq, 32)	ReLU + Dropout
BiLSTM block	BiLSTMCell	(128,)	ReLU + Dropout

Table 8
Hyperparameter configuration of the temporal feature extraction module.

Hyperparameter	Value
Input_dimension	32
Attention_head	4
FeedForward_hidden_size	64
BiLSTM_hidden_size	64

Table 9
Network structure of the classification module.

Layer	Output	Processing Fuction
Concat	(256,)	Concat
FC-4	(128,)	ReLU + Dropout
FC-5	(64,)	ReLU + Dropout
FC-6	(5,) for NSL-KDD, (7,) for CIC-IDS 2017, and (6,) for MQTTset	softmax

Table 10
Hyperparameter configuration of classification module.

Hyperparameter	Value
Batch_Size	256
Learning_Rate	0.0001
Dropout	0.5
Input_dimension	256

		Predicted	
		Positive	Negative
Actual	Positive	TP	FN
	Negative	FP	TN

Fig. 9. Confusion matrix.

- (2) Precision: It is the proportion of the number of correctly classified positive samples in all classified positive samples.

$$Precision = \frac{TP}{TP + FP} \quad (10)$$

- (3) Recall: It represents the proportion of all TP correctly classified and measures the ability of the classifier to recognize TP.

$$Sensitivity = \frac{TP}{TP + FN} \quad (11)$$

- (4) F1-score: It can be regarded as a weighted average of the accuracy and recall.

$$F1 - score = 2 \cdot \frac{precision \cdot recall}{precision + recall} \quad (12)$$

4.4. Performance evaluation of the NSL-KDD data set

We use the Res-TranBiLSTM model to conduct multiclassification experiments on the NSL-KDD data set. We introduce the detailed simulation results and verifies the effectiveness of data preprocessing through comparative experiments. The multiclassification results obtained by training and testing the original data set and the preprocessed data set are presented in Table 11. Furthermore, the graphical representation is depicted in Fig. 10.

From Table 11, it is evident found that the data set preprocessed using the SMOTE-ENN algorithm for data balancing significantly improves the recognition performance of classes with a small number of samples compared to the original data set, thereby verifying the robustness of this method. In terms of precision, R2L increased from 53.18% to 92.23%, Probe increased from 67.38% to 94.46%, U2R increased from 72.73% to 94.88%, Normal and DoS also increased by 5.57% and 4.67%, respectively. In terms of recall, U2R increased from 4.00% to 87.05%, R2L increased from 3.34% to 79.84%, and Probe, Normal, and DoS also increased by 20.45%, 4.47%, and 2.87%, respectively. With regard to the F1-score, U2R increased from 7.58% to 90.80%; R2L increased from 6.29% to 85.59%; and Probe, Normal, and DoS also increased by 23.95%, 5.18%, and 3.79%, respectively.

In addition, it was found that the Res-TranBiLSTM model has the lowest recognition accuracy for the Normal class, thereby indicating a relatively high false alarm rate largely due to the concealment of the original features.

4.5. Performance evaluation of the CIC-IDS2017 data set

In order to avoid certain problems inherent in the NSL-KDD data set mentioned above, we use the CIC-IDS2017 data set for further experiments. We introduce the simulation results in detail and verifies the effectiveness of data preprocessing through comparative experiments. The multiclassification results obtained by training and testing on the original data set and the balanced data set are presented in Table 12. Furthermore, the graphical representation is depicted in Fig. 11.

It is evident from Table 12 that the data set processed by class reorganization and SMOTE-ENN algorithm for data balancing significantly improves recognition performance, thereby verifying the robustness of this method. In terms of precision, Bot increased from 0.89% to 99.68%, Web Attack increased from 4.77% to 98.53%, Brute Force increased from 30.59% to 99.79%, and PortScan and DDoS also increased by 16.89% and 10.00%, respectively, DoS remains unchanged, and Normal decreased by 0.04%. In terms of recall, Bot increased from 32.94% to 99.97%; Web Attack increased from 45.71% to 98.87%; and Brute Force, DDoS, DoS, PortScan, and Normal also increased by 26.66%, 15.93%, 8.89%, 8.33% and 4.06%, respectively. With regard to F1-score, Bot increased from 1.73% to 99.82%; Web Attack increased from 8.64% to 98.70%; Brute Force is increased from 43.04% to 99.50%; and DDoS, PortScan, DoS, and Normal also increased by 13.07%, 12.86%, 4.68% and 2.06%, respectively.

In addition, it was found that the recognition accuracy of the Res-TranBiLSTM model for the class DoS is 100%, both for the preprocessed data and raw data. In addition to its own features that are easy to recognize, this also indicates that the combination of spatial and

temporal features has better recognition performance.

4.6. Performance evaluation of the MQTTset data set

The MQTT protocol is the most frequently used to communicate data in the IoT environment [56]. Therefore, in order to verify the generalization ability of the proposed model in IoT scenarios, it is necessary to include the dataset related to the MQTT protocol in the experiment. We used the MQTTset data set, which is created with MQTT protocol that is intended for IDS in IoT. We introduce the simulation results in detail and verifies the effectiveness of data preprocessing through comparative experiments. The multiclassification results obtained by training and testing on the original data set and the balanced data set are presented in Table 13. Furthermore, the graphical representation is depicted in Fig. 12.

From Table 13, it is evident found that the data set preprocessed using the SMOTE-ENN algorithm for data balancing significantly improves the recognition performance of classes with a small number of samples compared to the original data set, thereby verifying the robustness of this method. In terms of precision, Malformed increased from 57.44% to 98.91%, Brute Force increased from 64.15% to 99.07%, Flood increased from 76.85% to 99.77%; Dos, SlowITe and Normal also increased by 8.51%, 6.22% and 5.25%, respectively. In terms of recall, Flood increased from 67.70% to 99.74%, Brute Force increased from 70.75% to 98.96%, Malformed increased from 77.76% to 99.32%; and Normal, Dos and SlowITe also increased by 9.13%, 7.67%, and 3.04%, respectively. With regard to the F1-score, Malformed increased from 66.07% to 99.11%, Brute Force increased from 67.29% to 99.01%, Flood increased from 71.99% to 99.76%; Dos, Normal and SlowITe also increased by 8.10%, 7.23% and 4.65%, respectively.

In addition, it was found that the recognition accuracy of the Res-TranBiLSTM model for the class SlowITe is 100% for the preprocessed data, which is increased by over 6% compared with the raw data. This also indicates that the combination of the SMOTE-ENN algorithm and the feature confusion has better recognition performance.

4.7. Comparison result of NSL-KDD and CIC-IDS2017 data sets with other methods

We use the processed image data sets in Section 4.1 to train and test the state-of-the-art intrusion detection methods based on deep learning, including LeNet5, VGG16, ResNet18, and ResNet18 + BiLSTM. A comparison of respective performance is presented in Table 14 and Fig. 13.

It is evident from Table 14 that the generalization ability of the proposed model is higher than other deep learning algorithms. The overall detection accuracy of the NSL-KDD data set and the CIC-IDS2017 data set was 90.99% and 99.15%, respectively. For the NSL-KDD data set, the accuracy of this model was 13.76 and 21.9 percentage points higher than the traditional CNN models LeNet5 and VGGNet, respectively; 10.26 percentage points higher than the ResNet model; and 7.89 percentage points higher than the ReNet + BiLSTM model, which also combines spatial and temporal features.

However, due to a few inherent problems with the NSL-KDD data set, its test set contains classes and feature distributions that have never appeared in the training set. There are certain differences in the

Table 11

Multiclass classification performance of the Res-TranBiLSTM model on the NSL-KDD data set.

Attack Types	Precision (%)		Recall (%)		F1-Score (%)	
	Original_D	Processed_D	Original_D	Processed_D	Original_D	Processed_D
Normal	74.36	79.93	91.72	96.19	82.13	87.31
DoS	90.78	95.45	93.75	96.62	92.24	96.03
Probe	67.38	94.46	74.56	95.01	70.78	94.73
R2L	53.18	92.23	3.34	79.84	6.29	85.59
U2R	72.73	94.88	4.00	87.05	7.58	90.80

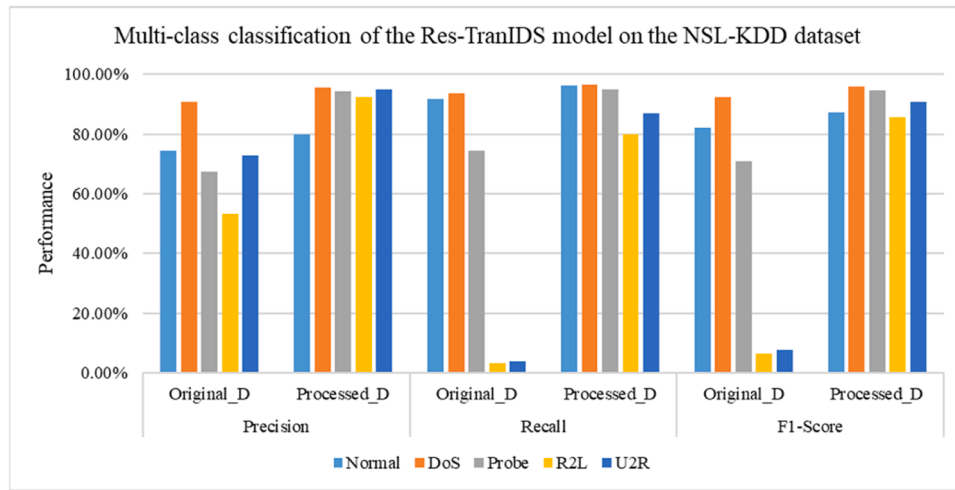


Fig. 10. Multiclass classification performance of the Res-TranBiLSTM model using the NSL-KDD data set.

Table 12

Multiclass classification performance of the Res-TranBiLSTM model on the CIC-IDS2017 data set.

Attack Types	Precision (%)		Recall (%)		F1-Score (%)	
	Original_D	Processed_D	Original_D	Processed_D	Original_D	Processed_D
Normal	99.94	99.90	95.83	99.89	97.84	99.90
Bot	0.89	99.68	32.94	99.97	1.73	99.82
Brute Force	30.59	99.79	72.56	99.22	43.04	99.50
DDoS	88.72	98.72	82.71	98.64	85.61	98.68
DoS	100.00	100.00	90.49	99.38	95.01	99.69
PortScan	80.50	97.39	89.70	98.03	84.85	97.71
Web Attack	4.77	98.53	45.71	98.87	8.64	98.70

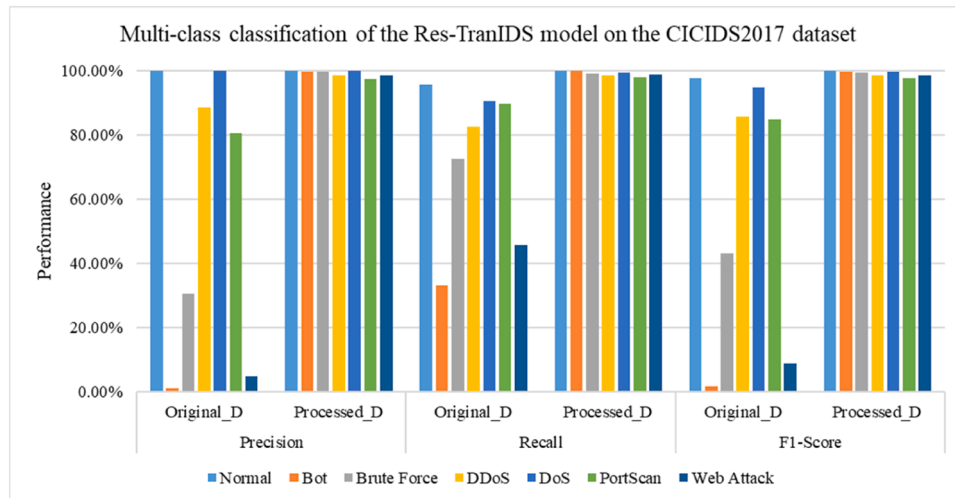


Fig. 11. Multiclass classification performance of the Res-TranBiLSTM model using the CIC-IDS2017 data set.

Table 13

Multiclass classification performance of the Res-TranBiLSTM model on the MQTTset data set.

Attack Types	Precision (%)		Recall (%)		F1-Score (%)	
	Original_D	Processed_D	Original_D	Processed_D	Original_D	Processed_D
Normal	94.62	99.87	90.66	99.79	92.60	99.83
DoS	91.15	99.67	92.11	99.78	91.63	99.72
Brute Force	64.15	99.07	70.75	98.96	67.29	99.01
Malformed	57.44	98.91	77.76	99.32	66.07	99.11
SlowITe	93.78	100.00	96.72	99.76	95.23	99.88
Flood	76.85	99.77	67.70	99.74	71.99	99.76

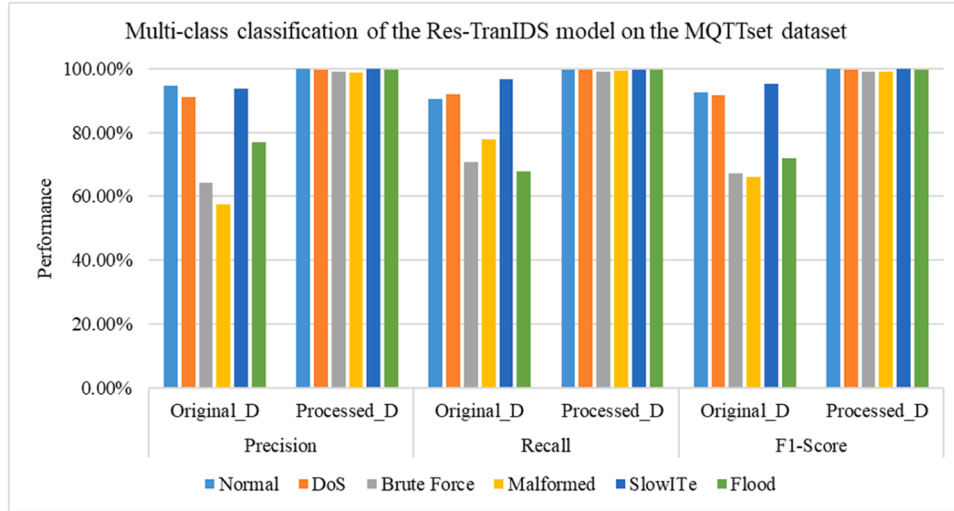


Fig. 12. Multiclass classification performance of the Res-TranBiLSTM model using the MQTTset data set.

Table 14
Accuracy comparison with different classification models.

Dataset	Model	Accuracy (%)
NSL-KDD	LeNet5	77.23
	VGG16	69.09
	ResNet18	80.73
	ResNet18 + BiLSTM	83.10
	Res-TranIDS (Ours)	90.99
CIC-IDS2017	LeNet5	83.51
	VGG16	72.40
	ResNet18	96.08
	ResNet18 + BiLSTM	96.77
	Res-TranIDS (Ours)	99.15

distribution of the training set and the test set, thereby resulting in generally low model accuracy. Therefore, in order to further explore the detection potential of the model, we find that the experimental results using the CIC-IDS2017 data set demonstrate better performance of the model, with an accuracy rate of 99.15%. In terms of accuracy, it is 15.64 and 26.75 percentage points higher than the traditional CNN models LeNet5 and VGGNet, respectively; 3.07 percentage points higher than

the ResNet model; and 2.38 percentage points higher than the ReNet + BiLSTM model.

First, in terms of spatial feature extraction, due to the problems of gradient disappearance and degradation in traditional CNN models, it is evident that VGGNet networks with deeper layers are not as effective as LeNet5 networks with shallower layers. On the other hand, networks with excessively shallow layers weakens their expressive ability. Therefore, using ResNet to extract spatial features can solve the above problems, and good results have been achieved using both data sets.

Second, for intrusion detection tasks, in addition to spatial features, the data set also contains temporal features. Therefore, the ResNet model—irrespective of whether we use traditional BiLSTM or the proposed TranBiLSTM to further extract temporal features—can improve accuracy.

Finally, in terms of temporal feature extraction, since the proposed Tran-BiLSTM model can achieve multimodal feature extraction using an attention-based encoder, it is evident that the accuracy of the proposed model on two data sets is 7.89 and 2.38 percentage points higher than that of the ResNet and BiLSTM models, respectively, with a significant improvement in the effect.

Furthermore, we compare Res-TranBiLSTM with other latest

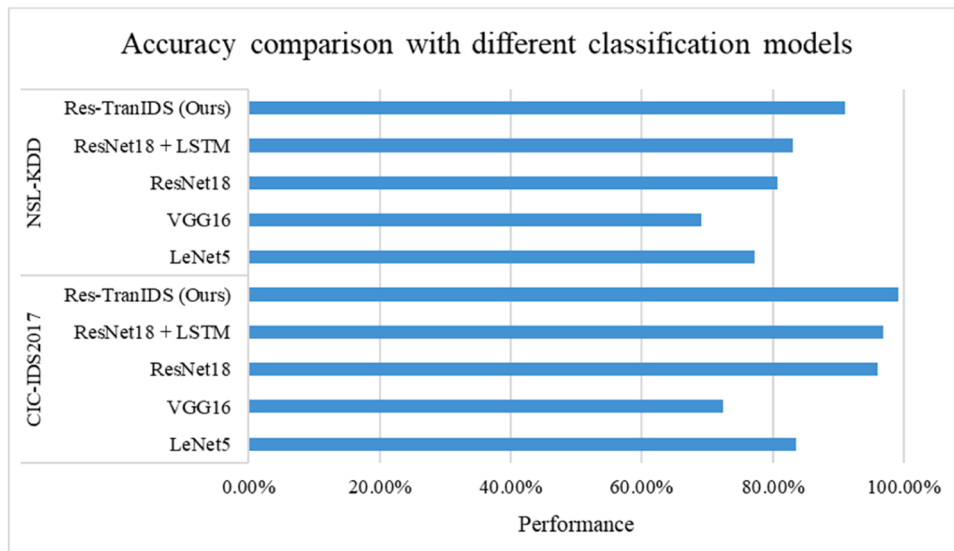


Fig. 13. Accuracy comparison with different classification models.

published models using the NSL-KDD and CIC-IDS2017 data sets. The comparison results are presented in Table 15. These methods include CNN, LSTM, GAN, etc., which are currently commonly used deep learning algorithms in IDS.

Table 15 indicates that the Res-TranBiLSTM model proposed in this paper outperforms the other related works that utilize both the NSL-KDD and CIC-IDS2017 data sets. On NSL-KDD dataset, we obtained the highest accuracy, recall, and F1-score, which were 90.99%, 90.94%, and 90.89%, respectively. It is worth noting that the precision of method [45] is slightly higher than our method. However, the recall of method [45] is more than 5 percentage points lower than our method. The recall can measure the degree to which the model can accurately identify relevant data, which cannot be ignored especially in IDS of medical IoT applications. On CIC-IDS2017 dataset, we all achieved optimal values.

4.8. Comparison result of MQTTset data set with other methods

In this section, we provide a comparison between the performance of our work with other state-of-the-art IDS models in IoT (using MQTTset data set). Some of them only focus on one or several types of attacks, some only perform binary classification, and some only use traditional ML algorithms. The comparison of achieved performance of the proposed Res-TranBiLSTM model and other existing models on the MQTTset data set is presented in Table 16. The scope of the comparison covers the IoT attacks, classifier type, ML model, and performance metrics.

Table 16 indicates that the Res-TranBiLSTM model proposed in this paper outperforms the other related works in IoT that utilize MQTTset data set. Our proposed model can perform classification and detection tasks for all 5 attacks and normal traffic in the dataset. In addition, we obtained the highest accuracy, precision, recall, and F1-score, which were 99.56%, 98.57%, 98.82%, and 99.55%, respectively. Compared to other multi-classification studies, it increased all indicators by more than 1 percentage point.

This fully proves that our proposed detection model considering both temporal and spatial features has certain reference value and further research value for the construction and optimization of large-scale IDSs in real IoT environments.

5. Conclusions and future directions

Intrusion detection is an important link in the field of information security protection for IoT and application systems and plays a positive role in protecting user information and intelligent devices. However, traditional intrusion detection systems based on statistics, expert systems, and ML methods have difficulty in meeting security requirements in the face of unprecedented network development and the continuous emergence of massive information.

In this study, we proposed an intrusion detection model called Res-TranBiLSTM. Aiming at the imbalance of original data, the SMOTE-ENN method was employed to alleviate the degree of data imbalance.

Then, a spatial feature extraction module based on ResNet and a temporal feature extraction module based on Transformer and BiLSTM were established to extract spatiotemporal features in parallel. Finally, spatiotemporal features were combined to achieve attack classification. We implement simulation experiments using Python programming language and Pytorch framework. Experimental results pertaining to the NSL-KDD and CIC-IDS2017 data sets revealed that the accuracy of the model classification reached 96.77%, thereby outperforming existing advanced methods. It increased the detection accuracy by about 1%–10% on NSL-KDD dataset and about 0.2%–10% on CIC-IDS2017 dataset. In addition, we provide a comparison between the performance of our work with other state-of-the-art IDS models in IoT using MQTTset data set. It increased all indicators by more than 1 percentage point. In detail, Res-TranBiLSTM not only improves overall detection performance but also improves detection performance for each attack class. Furthermore, the proposed model has certain reference value for the construction and optimization of large-scale IDS models, particularly in the IoT environment.

However, there are still certain aspects of Res-TranBiLSTM that are worth further research and improvement. First, due to the instability of the data captured in the real network environment, the proposed model is only conducted in simulation experiments. In the future, it is necessary to collect and analyze real IoT data based on specific application fields; moreover, it is also important to find suitable cleaning and preprocessing methods for existing network traffic information to complete the training and testing of the model in real network traffic. Second, with regard to spatial feature extraction, it is possible to explore more suitable methods for converting network traffic data to 2D images in order to improve image recognition, which will greatly improve the recognition level of the entire system. Third, it is necessary to consider how to combine these data for clustering and directly use the unlabeled traffic data obtained from the network to train the model. This not only achieves real-time classification detection but also real-time updates to the model when there is unknown new traffic, thereby achieving unsupervised learning.

CRediT authorship contribution statement

Shiyu Wang: Conceptualization, Methodology, Writing – original draft, Funding acquisition, Formal analysis. **Wenxiang Xu:** Conceptualization, Methodology, Investigation, Writing – original draft. **Yiwen Liu:** Conceptualization, Methodology, Writing – review & editing, Supervision.

Declaration of Competing Interest

We declare that we do not have any commercial or associative interest that represents a conflict of interest in connection with the work submitted.

Table 15

Comparison with other published methods using the NSL-KDD and CIC-IDS2017 data sets.

Dataset	Method	Data Source	Classifier Type	Model	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)
NSL-KDD	[35]	SF	MC	CNN	84.08	–	80.00	–
	[41]	SF	MC	CNN	80.51	–	80.78	–
	[57]	SF	MC	Hybrid RNN	82.91	86.78	82.91	80.41
	[45]	SF	MC	BiLSTM	89.60	93.10	85.30	89.40
	Ours	SF	MC	ResNet+TranBiLSTM	90.99	91.39	90.94	90.89
CIC-IDS2017	[38]	SF	BC	CNN	98.01	–	–	95.20
	[43]	SF + RT	MC	CNN	97.51	–	99.00	–
	[46]	SF	MC	CNN+LSTM	98.96	–	96.13	–
	[58]	SF	MC	BiGAN	82.30	76.50	76.30	76.40
	Ours	SF	MC	ResNet+TranBiLSTM	99.15	99.15	99.14	99.14

Note: SF: Statistic Features; RT: Raw Traffic; MC: Multi-Classification; BC: Binary Classification; “–”: no value is reported in the reference paper.

Table 16

Comparison with other published methods using the MQTTset data sets.

Method	IoT attack	Classifier Type	Model	Performance
[59]	Normal, DoS, BF, Malformed, SlowTTe, Flood	MC	DNN, CNN, RNN	Accuracy: 90.06% (DNN), 89.77% (CNN), 89.29% (RNN)
[60]	Normal, DoS, BF, Malformed, SlowTTe, Flood	MC	DNN, CNN, RNN	Accuracy: 90.40% (DNN), 90.76% (CNN), 90.05% (RNN)
[61]	Normal, DoS, BF, Malformed, SlowTTe, Flood	MC	ECSSA + LightGBM	Accuracy: 98.35%; Precision: 97.38%; Recall: 97.68%; F1-score: 98.56%
[62]	Normal, DoS, BF	BC	PPO + DNN + k-means	F1-score: 99.73%
[63]	Normal, DoS, BF, Malformed, SlowTTe, Flood	BC	Ensemble learning	Accuracy: 95.43%; F1-score: 95.42%
[64]	Normal, DoS, BF, Malformed, SlowTTe, Flood	BC	Ensemble learning	Accuracy: 91.59%
[65]	DoS, BF, Malformed	BC	Traditional ML	DoS (RF): Accuracy: 95.94%; Precision: 96.03%; Recall: 95.93% BF (DT): Accuracy: 99.81%; Precision: 99.81%; Recall: 99.81% Malformed (NN): Accuracy: 99.49%; Precision: 98.77%; Recall: 98.77%
Ours	Normal, DoS, BF, Malformed, SlowTTe, Flood	MC	ResNet+TranBiLSTM	Accuracy: 99.56%; Precision: 98.57%; Recall: 98.82%; F1-score: 99.55%

Note: BF: Brute Force; MC: Multi-Classification; BC: Binary Classification; ML: Machine Learning; RF: Random Forest; DT: Decision Tree; NN: Neural Network.

Data availability

The authors do not have permission to share data.

References

- [1] L. Vu, V. Cao, Q.U. Nguyen, D.N. Nguyen, D.T. Hoang, E. Dutkiewicz, Learning latent representation for IoT anomaly detection, *IEEE Trans. Cybern.* 52 (2022) 3769–3782, <https://doi.org/10.1109/TCYB.2020.3013416>.
- [2] N. Gupta, V. Jindal, P. Bedi, LIO-IDS: handling class imbalance using LSTM and improved one-vs-one technique in intrusion detection system, *Comput. Netw.* 192 (2021), 108076, <https://doi.org/10.1016/j.comnet.2021.108076>.
- [3] M. Almiani, A. AbuGhazleh, A. Al-Rahayfeh, S. Atiewi, A. Razaque, Deep recurrent neural network for IoT intrusion detection system, *Simul. Model. Pract. Theory* 101 (2020), <https://doi.org/10.1016/j.simpat.2019.102031>.
- [4] Y. Zhang, Q. Liu, On IoT intrusion detection based on data augmentation for enhancing learning on unbalanced samples, *Future Gen. Comput. Syst. Int. J. eScience* 133 (2022) 213–227, <https://doi.org/10.1016/j.future.2022.03.007>.
- [5] D. Li, L. Deng, M. Lee, H. Wang, IoT data feature extraction and intrusion detection system for smart cities based on deep migration learning, *Int. J. Inf. Manag.* 49 (2019) 533–545, <https://doi.org/10.1016/j.ijinfomgt.2019.04.006>.
- [6] H.C. Altunay, Z. Albayrak, A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks, *Eng. Sci. Technol. Int. J.* 38 (2023), 101322, <https://doi.org/10.1016/j.jestech.2022.101322>.
- [7] B. Mukherjee, S.J. Wang, W.Y. Lu, R.L. Neupane, D. Dunn, Y.J. Ren, Q. Su, P. Calyam, Flexible IoT security middleware for end-to-end cloud-fog communication, *Future Gen. Comput. Syst. Int. J. eScience* 87 (2018) 688–703, <https://doi.org/10.1016/j.future.2017.12.031>.
- [8] X. Zhao, G.Q. Huang, L. Gao, M.Z. Li, Q.L. Gao, Low load DIDS task scheduling based on Q-learning in edge computing environment, *J. Netw. Comput. Appl.* 188 (2021), <https://doi.org/10.1016/j.jnca.2021.103095>.
- [9] S. Javanmardi, M. Shojafar, R. Mohammadi, A. Nazari, V. Persico, A. Pescapè, FUSE: a security driven task scheduling approach for SDN-based IoT-Fog networks, *J. Inf. Secur. Appl.* 60 (2021), 102853, <https://doi.org/10.1016/j.jisa.2021.102853>.
- [10] X. Zhao, G.Q. Huang, J. Jiang, L. Gao, M.Z. Li, Task offloading of cooperative intrusion detection system based on Deep Q Network in mobile edge computing, *Expert Syst. Appl.* 206 (2022), <https://doi.org/10.1016/j.eswa.2022.117860>.
- [11] S. Javanmardi, M. Shojafar, R. Mohammadi, V. Persico, A. Pescapè, S-FoS: a secure workflow scheduling approach for performance optimization in SDN-based IoT-Fog networks, *J. Inf. Secur. Appl.* 72 (2023), <https://doi.org/10.1016/j.jisa.2022.103404>.
- [12] K. Kim, M.E. Aminanto, Deep learning in intrusion detection perspective: overview and further challenges, in: *Proceedings of the International Workshop on Big Data and Information Security (IWBIS), IEEE, 2017*, pp. 5–10.
- [13] D. Gumusbas, T. Yildirim, A. Genovese, F. Scotti, A comprehensive survey of databases and deep learning methods for cybersecurity and intrusion detection systems, *IEEE Syst. J.* 15 (2021) 1717–1731, <https://doi.org/10.1109/JSYST.2020.2992966>.
- [14] A.L. Buczak, E. Guven, A survey of data mining and machine learning methods for cyber security intrusion detection, *IEEE Commun. Surv. Tutor.* 18 (2016) 1153–1176, <https://doi.org/10.1109/COMST.2015.2494502>.
- [15] W. Zhong, N. Yu, C. Ai, Applying big data based deep learning system to intrusion detection, *Big Data Min. Anal.* 3 (2020) 181–195, <https://doi.org/10.26599/BDMA.2020.9020003>.
- [16] I.A. Kandhro, S.M. Alanazi, F. Ali, A. Kehar, K. Fatima, M. Uddin, S. Karuppayah, Detection of real-time malicious intrusions and attacks in IoT empowered cybersecurity infrastructures, *IEEE Access* 11 (2023) 9136–9148, <https://doi.org/10.1109/ACCESS.2023.3238664>.
- [17] J. Lansky, S. Ali, M. Mohammadi, M.K. Majeed, S.H.T. Karim, S. Rashidi, M. Hosseinzadeh, A.M. Rahmani, Deep learning-based intrusion detection systems: a systematic review, *IEEE Access* 9 (2021) 101574–101599, <https://doi.org/10.1109/ACCESS.2021.3097247>.
- [18] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural Comput.* 9 (1997) 1735–1780, <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [19] Y. Lecun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition, *Proc. IEEE* 86 (1998) 2278–2324, <https://doi.org/10.1109/5.726791>.
- [20] A. Krizhevsky, I. Sutskever, G.E. Hinton, ImageNet classification with deep convolutional neural networks, in: *Proceedings of the 25th International Conference on Neural Information Processing Systems, Lake Tahoe, Nevada, Curran Associates Inc., 2012*, pp. 1097–1105. Volume 1.
- [21] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, (2014). 10.48550/arXiv.1409.1556.
- [22] C. Szegedy, L. Wei, J. Yangqing, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1–9.
- [23] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [24] Y.H. Xiao, C. Xing, T.N. Zhang, Z.K. Zhao, An intrusion detection model based on feature reduction and convolutional neural networks, *IEEE Access* 7 (2019) 42210–42219, <https://doi.org/10.1109/ACCESS.2019.2904620>.
- [25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, *arXiv*, 2017.
- [26] S.H. Huang, Y. Liu, C. Fung, R. He, Y.N. Zhao, H.L. Yang, Z.Z. Luan, HitAnomaly: hierarchical transformers for anomaly detection in system log, *IEEE Trans. Netw. Serv. Manag.* 17 (2020) 2064–2076, <https://doi.org/10.1109/TNSM.2020.3034647>.
- [27] Z.H. Wu, H. Zhang, P.H. Wang, Z.B. Sun, RTIDS: a robust transformer-based approach for intrusion detection system, *IEEE Access* 10 (2022) 64375–64387, <https://doi.org/10.1109/ACCESS.2022.3182333>.
- [28] J.J. Hopfield, Neural networks and physical systems with emergent collective computational abilities, *Proc. Natl. Acad. Sci.* 79 (1982) 2554–2558, <https://doi.org/10.1073/pnas.79.8.2554>.

- [29] C.Y. Xu, J.Z. Shen, X. Du, F. Zhang, An intrusion detection system using a deep neural network with gated recurrent units, *IEEE Access* 6 (2018) 48697–48707, <https://doi.org/10.1109/ACCESS.2018.2867564>.
- [30] Y. Zhang, X. Chen, L. Jin, X.J. Wang, D. Guo, Network intrusion detection: based on deep hierarchical network and original flow data, *IEEE Access* 7 (2019) 37004–37016, <https://doi.org/10.1109/ACCESS.2019.2905041>.
- [31] S.Z. Wang, C.H. Xia, T.B. Wang, A novel intrusion detector based on deep learning hybrid methods, in: *Proceedings of the IEEE International Conference on Big Data Security on Cloud (BigDataSecurity), High Performance and Smart Computing (HPSC) and Intelligent Data and Security (IDS)*, IEEE, 2019, pp. 300–305.
- [32] Y. Zhang, X. Chen, D. Guo, M. Song, Y.L. Teng, X.J. Wang, PCCN: parallel cross convolutional neural network for abnormal network traffic flows detection in multi-class imbalanced network traffic flows, *IEEE Access* 7 (2019) 119904–119916, <https://doi.org/10.1109/ACCESS.2019.2933165>.
- [33] A. Kim, M. Park, D.H. Lee, AI-IDS: application of deep learning to real-time web intrusion detection, *IEEE Access* 8 (2020) 70245–70261, <https://doi.org/10.1109/ACCESS.2020.2986882>.
- [34] C. Xu, J. Shen, X. Du, A method of few-shot network intrusion detection based on meta-learning framework, *IEEE Trans. Inf. Forensics Secur.* 15 (2020) 3540–3552, <https://doi.org/10.1109/TIFS.2020.2991876>.
- [35] Z.Q. Hu, L.J. Wang, L. Qi, Y.M. Li, W.Z. Yang, A novel wireless network intrusion detection method based on adaptive synthetic sampling and an improved convolutional neural network, *IEEE Access* 8 (2020) 195741–195751, <https://doi.org/10.1109/ACCESS.2020.3034015>.
- [36] S. Sivamohan, S.S. Sridhar, S. Krishnaveni, An effective recurrent neural network (RNN) based intrusion detection via bi-directional long short-term memory, in: *Proceedings of the International Conference on Intelligent Technologies (CONIT)*, 2021, pp. 1–5.
- [37] O.M.A. Alsaibani, E. Utami, A.D. Hartanto, An intrusion detection system model based on bidirectional LSTM, in: *Proceedings of the 3rd International Conference on Cybernetics and Intelligent Systems (ICORIS)*, IEEE, 2021, pp. 13–18.
- [38] G. Andresini, A. Appice, D. Malerba, Nearest cluster-based intrusion detection through convolutional neural networks, *Knowl. Based Syst.* 216 (2021), <https://doi.org/10.1016/j.knosys.2021.106798>.
- [39] W.P. Wang, Z.R. Wang, Z.F. Zhou, H.X. Deng, W.L. Zhao, C.Y. Wang, Y.Z. Guo, Anomaly detection of industrial control systems based on transfer learning, *Tsinghua Sci. Technol.* 26 (2021) 821–832, <https://doi.org/10.26599/TST.2020.9010041>.
- [40] A. Fatani, M. Abd Elaziz, A. Dahou, M.A.A. Al-Qaness, S.F. Lu, IoT intrusion detection system using deep learning and enhanced transient search optimization, *IEEE Access* 9 (2021) 123448–123464, <https://doi.org/10.1109/ACCESS.2021.3109081>.
- [41] M. Sun, N. Liu, M. Gao, Research on intrusion detection method based on deep convolutional neural network, in: Q. Liang, W. Wang, J. Mu, X. Liu, Z. Na (Eds.), *Artificial Intelligence in China*, Springer Singapore, Singapore, 2022, pp. 537–544.
- [42] B. Deore, S. Bhosale, Hybrid optimization enabled robust CNN-LSTM technique for network intrusion detection, *IEEE Access* 10 (2022) 65611–65622, <https://doi.org/10.1109/ACCESS.2022.3183213>.
- [43] J.C. Yang, H.W. Li, S. Shao, F.T. Zou, Y. Wu, FS-IDS: a framework for intrusion detection based on few-shot learning, *Comput. Secur.* 122 (2022), <https://doi.org/10.1016/j.cose.2022.102899>.
- [44] J. Figueiredo, C. Serrão, A.M. de Almeida, Deep learning model transposition for network intrusion detection systems, *Electronics* 12 (2023) 293, <https://doi.org/10.3390/electronics12020293>.
- [45] B. Tang, Y. Lu, Q. Li, Y.Y. Bai, J. Yu, X. Yu, A diffusion model based on network intrusion detection method for industrial cyber-physical systems, *Sensors* 23 (2023), <https://doi.org/10.3390/s23031141>.
- [46] Y.N. Rao, K.S. Babu, An imbalanced generative adversarial network-based approach for network intrusion detection in an imbalanced dataset, *Sensors* 23 (2023), <https://doi.org/10.3390/s23010550>.
- [47] J. Ni, K. Zhang, X. Lin, X. Shen, Securing fog computing for internet of things applications: challenges and solutions, *IEEE Commun. Surv. Tutor.* 20 (2018) 601–628, <https://doi.org/10.1109/COMST.2017.2762345>.
- [48] C.A. de Souza, C.B. Westphall, R.B. Machado, L. Loffi, C.M. Westphall, G. A. Geronimo, Intrusion detection and prevention in fog based IoT environments: a systematic literature review, *Comput. Netw.* 214 (2022), 109154, <https://doi.org/10.1016/j.comnet.2022.109154>.
- [49] N.V. Chawla, K.W. Bowyer, L.O. Hall, W.P. Kegelmeyer, SMOTE: synthetic minority over-sampling technique, *J. Artif. Int. Res.* 16 (2002) 321–357.
- [50] D.L. Wilson, Asymptotic properties of nearest neighbor rules using edited data, *IEEE Trans. Syst. Man Cybern.* SMC-2 (1972) 408–421, <https://doi.org/10.1109/TSMC.1972.4309137>.
- [51] M. Tavallae, E. Bagheri, W. Lu, A.A. Ghorbani, A detailed analysis of the KDD CUP 99 data set, in: *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications*, 2009, pp. 1–6.
- [52] KDD Cup 1999 Data, 1999.
- [53] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli, E. Cambiaso, MQTTset, a new dataset for machine learning techniques on MQTT, *Sensors* 20 (2020) 6578, <https://doi.org/10.3390/s20226578>.
- [54] S. Ghazanfar, F. Hussain, A.U. Rehman, U.U. Fayyaz, F. Shahzad, G.A. Shah, IoT-flock: an open-source framework for IoT traffic generation, in: *Proceedings of the International Conference on Emerging Trends in Smart Technologies (ICETST)*, 2020, pp. 1–6.
- [55] A.H. Lashkari, A. Seo, G.D. Gil, A. Ghorbani, CIC-AB: online ad blocker for browsers, in: *Proceedings of the International Carnahan Conference on Security Technology (ICCST)*, 2017, pp. 1–7.
- [56] P.K. Donta, S.N. Srirama, T. Amgoth, C.S.R. Annavarapu, Survey on recent advances in IoT application layer protocols and machine learning scope for research directions, *Digit. Commun. Netw.* 8 (2022) 727–744, <https://doi.org/10.1016/j.dcan.2021.10.004>.
- [57] P.B. Udas, M.E. Karim, K.S. Roy, SPIDER: a shallow PCA based network intrusion detection system with enhanced recurrent neural networks, *J. King Saud Univ. Comput. Inf. Sci.* 34 (2022) 10246–10272, <https://doi.org/10.1016/j.jksuci.2022.10.019>.
- [58] W. Yao, H. Shi, H. Zhao, Scalable anomaly-based intrusion detection for secure Internet of Things using generative adversarial networks in fog environment, *J. Netw. Comput. Appl.* 214 (2023), <https://doi.org/10.1016/j.jnca.2023.103622>.
- [59] M.A. Ferrag, O. Friha, L. Maglaras, H. Janicke, L. Shu, Federated deep learning for cyber security in the internet of things: concepts, applications, and experimental analysis, *IEEE Access* 9 (2021) 138509–138542, <https://doi.org/10.1109/ACCESS.2021.3118642>.
- [60] O. Friha, M.A. Ferrag, L. Shu, L. Maglaras, K.K.R. Choo, M. Nafaa, FELIDS: federated learning-based intrusion detection system for agricultural Internet of Things, *J. Parallel Distrib. Comput.* 165 (2022) 17–31, <https://doi.org/10.1016/j.jpdc.2022.03.003>.
- [61] C. Prajisha, A.R. Vasudevan, An efficient intrusion detection system for MQTT-IoT using enhanced chaotic salp swarm algorithm and LightGBM, *Int. J. Inf. Secur.* 21 (2022) 1263–1282, <https://doi.org/10.1007/s10207-022-00611-9>.
- [62] H. Han, H. Kim, Y. Kim, Correlation between deep neural network hidden layer and intrusion detection performance in IoT intrusion detection system, *Symmetry* 14 (2022), <https://doi.org/10.3390/sym14102077>.
- [63] H. Zeghida, M. Boulaiche, R. Chikh, Securing MQTT protocol for IoT environment using IDS based on ensemble learning, *Int. J. Inf. Secur.* (2023), <https://doi.org/10.1007/s10207-023-00681-3>.
- [64] S. Hariprasad, T. Deepa, An ensemble intrusion detection system based on acute feature selection, *Multimed. Tools Appl.* (2023), <https://doi.org/10.1007/s11042-023-15788-x>.
- [65] Imran, M.F. Zuhairi, S.M. Ali, Z. Shahid, M.M. Alam, M.M. Su'ud, Improving reliability for detecting anomalies in the MQTT network by applying correlation analysis for feature selection using machine learning techniques, *Appl. Sci.* 13 (2023), <https://doi.org/10.3390/app13116753>.



Shiyu Wang received the bachelor's degree in Information Management and Information System from Shandong Normal University, Shandong, China, in 2018 and the M.S. degree in Control Science and Engineering from Shanghai University of Engineering Science, Shanghai, China, in 2022. She is currently working in Research Institute of China Telecom. Her research interests include cybersecurity, intrusion detection, and artificial intelligence.



Wenxiang Xu received the bachelor's degree in Measurement and Control Technology and Instrumentation Program from Liaoning University, Liaoning, China, in 2018 and the M.S. degree in Detection Technology and Automatic Equipment from University of Chinese Academy of Sciences, Beijing, China, in 2021. She is currently working in Research Institute of China Telecom. Her research interests include application security and API security.



Yiwen Liu received the bachelor's degree in Automation from Shanghai Polytechnic University, Shanghai, China, in 2018 and the M.S. degree in Control Science and Engineering from Shanghai University of Engineering Science, Shanghai, China, in 2021. She is currently pursuing the Ph.D. degree with the Control Science and Engineering, Donghua University, Shanghai, China. Her research interests include deep learning and computer vision.